

Platt Scaling

2026-04-17

Introduction

Platt scaling (Platt, 1999) is a post-hoc calibration method that maps a classifier's raw scores onto well-calibrated probabilities by fitting a logistic regression from scores to true labels on a held-out validation set. It is the simplest and often best calibration method for SVMs, neural networks, and other classifiers whose mis-calibration is approximately monotone. The idea is to leave the classifier's ranking untouched while rescaling the score axis so that the predicted probability of class 1 matches the empirical class-1 rate.

Prerequisites

A working understanding of calibration plots, the difference between discrimination and calibration, logistic regression, and the use of held-out validation sets.

Theory

Given validation pairs (f_i, y_i) where f_i is the classifier score (decision value, logit, or raw probability), fit

$$P(y = 1 \mid f) = \frac{1}{1 + \exp(Af + B)},$$

by maximum likelihood. New predictions are $P(y = 1 \mid f(x))$ from this fitted sigmoid. The transformation has only two parameters and preserves the rank order — and therefore the ROC curve and AUC — of the original classifier. Label smoothing $(y + 1)/(N + 2)$ avoids numerical issues near 0 and 1 when the validation set is small.

Assumptions

Mis-calibration is monotone (a sigmoid family is appropriate); a separate validation set is available for fitting the calibration; the classifier's scores rank observations meaningfully (Platt scaling cannot fix a classifier with poor discrimination).

R Implementation

```
set.seed(2026)
n <- 1000
# Well-ranked but mis-calibrated scores
y <- rbinom(n, 1, 0.3)
f <- 2 * y + rnorm(n, 0, 1.2)      # decision function

# Naive sigmoid of f does not match observed frequencies
naive_p <- plogis(f)

# Platt scaling: fit logistic regression y ~ f on validation
val_idx <- 1:500
te_idx <- 501:1000
```

```
fit <- glm(y[val_idx] ~ f[val_idx], family = "binomial")
A <- coef(fit)[2]; B <- coef(fit)[1]
platt_p <- plogis(A * f[te_idx] + B)

# Brier scores
c(naive = mean((naive_p[te_idx] - y[te_idx])^2),
  platt = mean((platt_p - y[te_idx])^2))
```

Output & Results

Brier score (mean squared error of probability vs label) drops after Platt scaling, reflecting improved calibration. AUC remains unchanged because the rescaling is monotone and preserves rank.

Interpretation

A reporting sentence: “Platt scaling reduced the Brier score from 0.21 (naive sigmoid of SVM decision values) to 0.18 (Platt-rescaled) on the held-out test set; ROC AUC was unchanged at 0.81 because the rescaling preserves rank order. The reliability diagram shifted toward the diagonal at every probability decile after recalibration.” Always report Brier score and a reliability diagram before and after calibration.

Practical Tips

- Platt scaling works best when mis-calibration is smooth and monotone; for wiggly or non-monotone calibration curves, isotonic regression is preferred.
- Use a separate validation set (not training data) to fit the calibration; in-sample calibration is self-reinforcing and biased.
- Platt scaling adds only 2 parameters and is extremely data-efficient — a few hundred validation points are usually sufficient.
- For multi-class problems, fit one-vs-rest Platt scaling per class and renormalise the resulting probabilities to sum to 1.
- Pair with a reliability diagram (`ggcalibrate` or hand-built) before and after calibration to verify the visual improvement.
- Platt scaling cannot improve discrimination; if AUC is poor, fix the underlying classifier rather than chasing better calibration.

R Packages Used

Base R `glm(family = "binomial")` for fitting the Platt sigmoid; `caret::calibrate()` for an integrated calibration workflow; `probably` for tidymodels-flavoured calibration utilities; `betacal` for Beta calibration as an alternative parametric form; `mlr3calibration` for calibration extensions in the `mlr3` ecosystem.

For Reviewers

What to look for in a paper using this method.

- Common misapplications.
- Diagnostics that should be reported but often aren’t.
- Red flags in tables and figures.
- What to verify.
- What an adequate Methods paragraph must contain.

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net

- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis